АСЭ
РОСАТОМ

Лекция 1

Vite, Vite+, Oxc и Nginx

Эрнандес Гарсия Даниэль Дельевич

Веб-разработчик

О чём лекция

Соберём инструментарий, с которым фронтенд-проект удобно запускать, проверять и доставлять до пользователя.

Vite отвечает за разработку и сборку: мгновенный старт, HMR и оптимизированный production-бандл.

Vite+ добавляет единый CLI: зависимости, линтинг, форматирование, тесты и задачи.

Oxc даёт быстрые линтер и форматтер на Rust.

Nginx принимает трафик в проде: статика, SPA-fallback, proxy и HTTPS.

Разберём, как всё устроено, какие настройки действительно важны и как выглядит рабочий конфиг.

АСЭ
РОСАТОМ

01

Vite

Dev-сервер, сборка и конфиг

Как Vite ускоряет разработку

В dev-режиме Vite не собирает весь проект заранее. Он отдаёт исходники по запросу как ESM: браузер сам подгружает модули, а Vite пересобирает только изменённое. Поэтому старт почти мгновенный, а HMR обновляет страницу без полной перезагрузки.

В production Vite собирает оптимизированный бандл: минифицирует код, разбивает его на чанки и подготавливает ассеты.

Команда	Что делает
npm create vite@latest	Новый проект из шаблона
npm install -D vite	Vite в существующий проект

Шаблоны: react, react-ts, vue, svelte, solid, preact, lit, vanilla.

Конфиг: файл и форма

Конфиг Vite живёт в vite.config.ts. Минимальный вариант — объект:

```
import { defineConfig } from 'vite';

export default defineConfig({});
```

Если настройки зависят от команды, экспортируйте функцию. Vite передаёт command: serve для vite и build для vite build.

```
export default defineConfig(({ command }) => ({
  base: command === 'build' ? '/app/' : '/',
}));
```

Форма	Когда нужна	Что получает
объект	статичные настройки	—
функция	разные serve/build	{ command, mode }

Почему .env не виден в конфиге

Vite выполняет конфиг раньше, чем читает .env: иначе он не знал бы envDir (откуда брать файлы) и envPrefix (что пускать в клиент). Поэтому в process.env внутри конфига — только переменные ОС. process.env — глобальный объект Node.js, process.cwd() — текущий рабочий каталог.

Значения из .env загружают вручную через loadEnv:

```
import { defineConfig, loadEnv } from 'vite';

export default defineConfig(({ mode }) => {
  const env = loadEnv(mode, process.cwd(), '');
  return {
    define: { __API__: JSON.stringify(env.VITE_API_URL) },
  };
});
```

Что	Когда читается	Что содержит
process.env	при выполнении конфига	переменные ОС
.env	после конфига	VITE_-переменные
import.meta.env	в коде приложения	клиентские значения

root и base: что это и когда менять

root — папка с index.html. По умолчанию — process.cwd(), каталог запуска команды. Если index.html в app/, укажите root: 'app', иначе dev-сервер его не найдёт.

base — путь приложения на домене. По умолчанию '/'; для деплоя в подпапку — '/app/'. Vite добавит префикс к путям и в import.meta.env.BASE_URL.

```
export default defineConfig({
  root: 'app',
  base: '/app/',
});
```

Опция	По умолчанию	Пример
root	process.cwd()	root: 'app'
base	'/'	base: '/app/'

В dev-сервере приложение откроется на localhost:5173/app/.

define: подстановка значений

АСЭ
РОСАТОМ

define заменяет идентификаторы на этапе сборки — как #define в C. В dev значение добавляется в globalThis, в сборке Vite меняет код на уровне AST. Строки оборачивайте в JSON.stringify, иначе подставится идентификатор, а не строка. Глобалы типизируют в vite-env.d.ts.

```
export default defineConfig({
  define: {
    __APP_VERSION__: JSON.stringify('1.0.0'),
  },
});
```

Режим	Что происходит
dev	значение в globalThis
build	подстановка в AST
строка	нужен JSON.stringify

TypeScript: объявите __APP_VERSION__ в vite-env.d.ts.

plugins и publicDir

plugins — массив плагинов. false, null и undefined Vite игнорирует, вложенные массивы распаковывает, Promise дожидается. Поэтому удобно писать isDev && plugin().

publicDir — статика, которая отдаётся как есть: favicon, robots.txt, картинки. По умолчанию public. В dev файлы доступны от /, в сборке копируются в dist. publicDir: false отключает и то, и другое.

```
export default defineConfig({
  plugins: [react()],
  publicDir: 'static',
});
```

Настройка	Что делает	По умолчанию
plugins	подключает плагины	[]
publicDir	статика без обработки	'public'
publicDir: false	отключает статику	—

Файл static/logo.png будет доступен как /logo.png.

resolve и служебные опции

АСЭ
РОСАТОМ

resolve.extensions позволяет опускать расширения в import; по умолчанию: .mjs, .js, .mts, .ts, .jsx, .tsx, .json — побеждает раннее. resolve.tsconfigPaths включает алиасы из compilerOptions.paths (по умолчанию false). clearScreen чистит терминал (true); отключают, чтобы не терять логи HMR. envDir — папка .env (root; false отключает), envPrefix — префикс клиентских переменных (VITE_); меняют, чтобы серверные секреты не попали в бандл.

```
export default defineConfig({
  resolve: {
    extensions: ['.ts', '.tsx', '.js'],
    tsconfigPaths: true,
  },
  clearScreen: false,
  envPrefix: 'APP_',
```

Опция	По умолчанию	Зачем
resolve.extensions	список расширений	опускать расширения
resolve.tsconfigPaths	false	алиасы из tsconfig
clearScreen	true	чистить терминал
envDir	root	папка .env
envPrefix	VITE_	клиентские переменные

Dev-сервер: порт, HTTPS и proxy

Порт по умолчанию — 5173; strictPort запрещает занимать другой. https включает TLS/HTTP2, open открывает браузер, cors настраивает CORS.

proxy перенаправляет API на бэкенд и обходит CORS, forwardConsole: true показывает консоль браузера в терминале.

```
export default defineConfig({
  server: {
    port: 3000,
    strictPort: true,
    proxy: { '/api': 'http://localhost:8080' },
    forwardConsole: true,
  },
});
```

Опция	По умолчанию	Что делает
port	5173	Порт dev-сервера
strictPort	false	Запрещает другой порт
https	false	TLS/HTTP2; key/cert или SSL-плагин
open	false	Браузер; строка — путь
cors	false	true — любой origin; объект
proxy	—	API на бэкенд, обход CORS
forwardConsole	false	Консоль браузера в терминал

Плагины и кеш зависимостей

Для React — @vitejs/plugin-react; для HTTPS — @vitejs/plugin-basic-ssl:

```
import basicSsl from '@vitejs/plugin-basic-ssl';

export default {
  plugins: [basicSsl()],
};
```

@vitejs/plugin-react включает Fast Refresh. Зависимости кешируются в node_modules/.vite — это ускоряет повторный старт; кеш можно удалить.

Что	Где	Зачем
react	plugins	React + Fast Refresh
basic-ssl	plugins	локальный HTTPS
кеш	node_modules/.vite	быстрый старт

Ассеты и стили

import imgUrl from './img.png' при сборке превращается в URL; используйте его как обычную ссылку, например в src тега <img>.

Для .scss, .sass, .less и .styl плагины не нужны — поставьте сам препроцессор:

```
npm install -D sass
```

Vite подхватит его и учтёт алиасы из конфига в @import для Sass/Less.

Что	Как
картинка	import imgUrl from './img.png'
препроцессор	npm i -D sass
алиасы	учитываются в @import

.env и режимы: что и когда загружается

АСЭ
РОСАТОМ

В клиент попадают только переменные с префиксом VITE_. Все значения в import.meta.env — строки, сравнивайте явно: VITE_F === 'true'. Без префикса — undefined. Префикс меняется опцией envPrefix.

```
export default defineConfig({
  envDir: './env',
  envPrefix: 'APP_',
});
```

Приоритет	Файл
1	.env.[mode].local
2	.env.[mode]
3	.env.local
4	.env

Встроенные поля: MODE — текущий режим (development или production), BASE_URL — базовый URL из base, PROD/DEV/SSR — булевы флаги продакшена, разработки и SSR. Кастомные переменные всегда строки, встроенные — boolean.

Типы для import.meta.env

import.meta — объект с метаданными ESM-модуля; Vite добавляет env. В TypeScript структуру описывает ImportMetaEnv: по умолчанию TS знает только MODE, DEV, PROD, BASE_URL, SSR. Остальные переменные объявите в src/vite-env.d.ts — иначе ошибка типа.

```
/// <reference types="vite/client" />

interface ImportMetaEnv {
  readonly VITE_APP_TITLE: string;
}

interface ImportMeta {
  readonly env: ImportMetaEnv;
}
```

Поле	Тип
MODE, BASE_URL, VITE_*	string
DEV, PROD, SSR	boolean
ImportMetaEnv	интерфейс для env

АСЭ
РОСАТОМ

02

Vite+

Единый CLI для зависимостей и проверок

Vite+: CLI и пакет — разные вещи

АСЭ
РОСАТОМ

vp — глобальный CLI, который вы запускаете в терминале. vite-plus — локальный пакет проекта: он даёт defineConfig, плагины и фиксирует версию инструментов. Без vp не запустятся команды, без vite-plus проект потеряет локальную версию.

```
import { defineConfig } from 'vite-plus';

export default defineConfig({
  plugins: [react()],
});
```

Что	Где	Зачем
vp	глобально	dev, build, check, fmt, test
vite-plus	dependencies	defineConfig, plugins, версия
vite.config.ts	проект	единый конфиг

vite.config.ts остаётся стандартным: Vite+ не вводит второй конфиг.

Vite+: зависимости и команды

Зависимости ставятся через vp. Менеджер пакетов задаётся в package.json полем devEngines.packageManager, а точная версия — полем packageManager, например bun@1.1.20.

Команда	Действие
vp add <pkg>	добавить пакет
vp add -D <pkg>	добавить dev-пакет
vp remove <pkg>	удалить пакет
vp update	обновить зависимости

vp dev — dev-сервер, vp build — сборка, vp upgrade — обновление глобального CLI. Для CI — voidzero-dev/setup-vp.

Проверки: check и lint

vp check — одна команда для форматирования, линтинга и проверки типов; флаг --fix включает автоисправление. Под капотом vp lint работает на Oxlint. У него есть typeCheck (запускает компилятор TypeScript во время линтинга) и typeAware (ищет логические ошибки на основе типов).

Конфиг линтера живёт в vite.config.ts, в блоке lint, а не в oxlint.config.ts:

```
export default defineConfig({
  lint: { ignorePatterns: ['dist/**'] },
});
```

vp lint запускает линтинг, vp lint --fix — с автоисправлением. Флаг --fix у vp check включает автоисправление сразу для форматтера и линтера.

Форматирование и тесты

АСЭ
РОСАТОМ

vp fmt форматирует код. По умолчанию он ничего не пишет в файлы: чтобы сохранить результат, нужно vp fmt . --write. Под капотом — Oxfmt.

Тесты запускаются командой vp test и работают на Vitest.

Оба конфига живут в vite.config.ts — в блоках fmt и test, а не в отдельных oxfmt.config.ts и vitest.config.ts.

Команда	Что делает
vp fmt	форматирование
vp fmt . --write	записать в файлы
vp test	тесты (Vitest)

Инструмент	Блок	Не в файле
fmt	fmt	oxfmt.config.ts
test	test	vitest.config.ts

Задачи и vpx

vp run <script-name> запускает скрипт из package.json. Разница с vp dev: vp dev — встроенная команда, а vp run dev запускает именно скрипт dev из package.json.

Задача — именованный скрипт. Задачи описываются в vite.config.ts, в блоке run.tasks; у задачи есть command и dependsOn — зависимость от другой задачи. Vite+ кеширует состояние задачи по входным и выходным файлам, поэтому повторный запуск может быть почти мгновенным.

```
run: {
  tasks: {
    build: { command: 'vp build', dependsOn: ['test'] },
  },
}
```

Запуск задачи — vp run <task-name>. Для разовых бинарников есть vpx — аналог npx и bunx.

Git-хуки

Хуки включаются командой vp hooks enable и выключаются через vp hooks disable; vp hooks status показывает текущее состояние.

Скрипты лежат в папке .vite-hooks в корне проекта. Её нужно коммитить, чтобы у всей команды были одинаковые хуки. Для pre-commit создаётся файл .vite-hooks/pre-commit с командой vp staged — она проверяет только изменённые файлы.

```
# .vite-hooks/pre-commit
vp staged
```

Через staged в vite.config можно задать свои команды: staged: { '*.{js,ts,tsx}': 'vp check --fix' }.

АСЭ
РОСАТОМ

03

Oxc

Линтер и форматтер на Rust

Oxc и Oxlint

Oxc — набор высокопроизводительных инструментов для JavaScript и TypeScript, написанных на Rust. Oxlint — его линтер на стеке Oxc: быстрый, совместимый с ESLint и использующий tsgo (TypeScript 7).

Установка и скрипты:

```
npm install -D oxlint


// package.json
"scripts": {
  "lint": "oxlint",
  "lint:fix": "oxlint --fix"
}
```

Oxlint проверяет код, а --fix автоматически исправляет то, что можно исправить. Запуск: npm run lint или npm run lint:fix.

Конфиг и плагины Oxlint

Конфиг создаётся командой oxlint --init. Файл называется .oxlintrc.json или oxlint.config.ts. Базовый вариант:

```
import { defineConfig } from 'oxlint';

export default defineConfig({});
```

Плагин — это именованная группа правил; его включение делает правила доступными. Встроенные плагины перечисляются в поле plugins, например plugins: ['import']. Эта запись переопределяет набор плагинов, включённых по умолчанию. Плагин eslint включён по умолчанию и подключает основные правила ESLint.

Oxc внутри Vite+

АСЭ
РОСАТОМ

В проекте на Vite+ отдельный .oxlintrc.json не нужен: Oxc настраивается прямо в vite.config.ts. Вот реальный фрагмент конфига:

```
import { defineConfig } from 'vite-plus';

export default defineConfig({
  lint: {
    ignorePatterns: ['dist/**', 'node_modules/**'],
    plugins: ['react', 'typescript', 'oxc'],
    rules: { 'react/rules-of-hooks': 'error' },
    options: { typeAware: true, typeCheck: true },
  },
  fmt: {},
});
```

Блок	Зачем
lint.plugins	react, typescript, oxc
lint.rules	точечные правила
lint.options	typeAware + typeCheck
fmt	настройки Oxfmt

vp check запускает всё сразу: Oxfmt, Oxlint и проверку типов.

АСЭ
РОСАТОМ

04

Nginx

Точка входа в production: статика, proxy и HTTPS

Зачем Nginx фронтенду

После сборки Vite получается папка dist с HTML, JS, CSS и шрифтами. Отдавать её через Node-сервер можно, но дороже и медленнее: Nginx делает это эффективнее и берёт на себя инфраструктурные задачи.

Статика — быстрая отдача файлов.

Reverse proxy — прячет API, снимает CORS и балансирует нагрузку между инстансами.

TLS и кэш — терминирует HTTPS, сжимает gzip и кэширует ассеты на клиенте.

В типичном деплое Nginx — точка входа, а Node/API — за ним.

Роль	Что делает
Статика	HTML, JS, CSS, шрифты
Reverse proxy	API, CORS, балансировка
TLS и кэш	HTTPS, gzip, кэш

Карта конфига и ключевые директивы

Конфиг Nginx вложен сверху вниз: настройки из http наследуются в server, а из server — в location. Простые директивы заканчиваются ;, блочные — { }. Подключение файлов — include /etc/nginx/conf.d/*.conf;.

Контекст	Что настраивает
main	user, worker_processes, error_log
events	worker_connections
http	mime.types, gzip, upstream
server	listen, server_name, root
location	try_files, proxy_pass

Для SPA важны try_files (fallback на index.html), для API — proxy_pass, для ассетов — expires и gzip, для HTTPS — ssl и return 301.

Минимальный конфиг SPA

Вот минимальный server, которого хватает для SPA: статика, fallback на index.html и proxy на API.

```
server {
  listen 80;
  server_name example.com;
  root /usr/share/nginx/html;


  location / {
    try_files $uri $uri/ /index.html;
  }
  location /api/ { proxy_pass http://api:3000/; }
  location ~* \.(js|css|png|svg|woff2)$ {
    expires 30d;
    add_header Cache-Control "public, immutable";
  }
}
```

Без try_files прямой заход на /posts/1 даст 404. gzip, кэш и HTTPS добавляются в этот же server. Проверка: nginx -t → nginx -s reload.

Итог

Vite — быстрый dev-сервер и сборщик; его конфиг живёт в vite.config.ts.

Vite+ — единый CLI vp для зависимостей, проверок, тестов и задач.

Oxc — Oxlint и Oxfmt на Rust для быстрого линтинга и форматирования.

Nginx — точка входа в проде: статика, SPA-fallback, proxy и HTTPS.

Дальше: собрать проект на Vite+, настроить vp check и развернуть его за Nginx.